

React-to-Angular Subset Compiler

Project Proposal

Project Title

React-to-Angular Subset Compiler

Introduction

Modern front-end development depends heavily on JavaScript frameworks such as React and Angular.

Each framework uses different architectures, syntaxes, and rendering approaches, which makes migrating applications between frameworks difficult and time-consuming.

This project proposes the design and implementation of a custom compiler capable of translating a subset of React components into Angular components automatically.

The compiler demonstrates the complete compilation pipeline including lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and target code generation.

Problem Statement

Companies and developers often face challenges when migrating applications from one framework to another.

For example:

-

A company may have an existing React codebase but wants to move to Angular.

-

Manual conversion requires significant time and effort.

-

Developers must rewrite components manually.

-

JSX syntax differs greatly from Angular templates.

-

Human conversion may introduce bugs and inconsistencies.

Currently, most migration processes are manual or semi-automatic, making framework migration expensive and error-prone.

This project solves this problem by introducing a mini compiler capable of automatically translating a subset of React syntax into Angular-compatible code.

Proposed Solution

The proposed solution is a Source-to-Source Compiler (Transpiler) that:

-

Reads React component code.

-

Performs lexical analysis to generate tokens.

-

Builds an Abstract Syntax Tree (AST).

-

Validates semantic correctness.

-

Generates an Intermediate Representation (IR).

-

Optimizes the generated representation.

-

Produces Angular component code automatically.

The compiler focuses on a controlled subset of React syntax to simplify parsing and

transformation while still demonstrating real compiler concepts.

Supported React Subset

The compiler currently supports:

- Function Components
- JSX Elements
- Nested JSX Structures
- Text Rendering
- Basic HTML Tags

Example Supported Input:

```
function App() {  
  return <h1>Hello World</h1>  
}
```

Generated Angular Output:

```
@Component({  
  selector: 'app-app',  
  template: `<h1>Hello World</h1>`  
})  
export class AppComponent {}
```

Compiler Phases

1. Lexical Analysis

The compiler converts source code into tokens such as keywords, identifiers, and JSX elements.

2. Syntax Analysis

The parser validates syntax correctness and constructs an Abstract Syntax Tree (AST).

3. Semantic Analysis

The semantic analyzer verifies logical correctness and validates component structure.

4. Intermediate Code Generation

The compiler generates an intermediate representation describing the component structure.

5. Code Optimization

Basic optimization techniques are applied to simplify generated templates.

6. Target Code Generation

The compiler produces Angular component code in TypeScript format.

Technologies Used

-

Python

-

Regular Expressions

-

Recursive Descent Parsing

-

Abstract Syntax Trees (AST)

Expected Outcomes

By the end of the project, the compiler will:

- Demonstrate all major compiler phases.
- Translate React subset syntax into Angular code.
- Generate valid Angular component structures.
- Provide detailed logs for every compiler stage.

Conclusion

This project combines compiler design concepts with modern front-end technologies. Instead of creating a traditional arithmetic compiler, the project introduces a practical real-world compiler use case by transforming React components into Angular components.

The project demonstrates both theoretical compiler knowledge and practical software engineering skills.